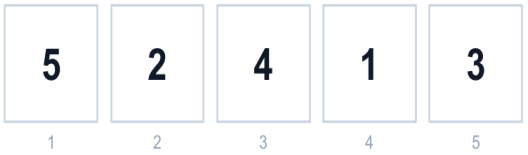
Sorting Algorithms

Case analysis on $O(N^2)$ comparison-based sorting algorithms

Bubble Sort • Selection Sort • Insertion Sort

Running example



Input size = N; here N = 5

Course focus

- Compare adjacent or selected elements
- Analyze best, average, worst cases
- Count iterations, comparisons, swaps/shifts
- Identify stable vs. unstable behavior

Formal analysis criteria

The same checklist will be applied to Bubble, Selection, and Insertion Sort.

Time complexity	Best / average / worst number of primitive operations as N grows.
Space complexity	Extra memory used beyond the input array. These three are in-place: O(1).
Stability	Equal keys keep their original relative order after sorting.
Adaptiveness	Algorithm becomes faster when the input is already or nearly sorted.
Data movement	Bubble uses swaps; Selection uses minimum swaps; Insertion uses shifts.

Stability: an academic definition

A stable sorting algorithm preserves the relative order of records with equal keys.

Before sorting by numeric key

3A	1	3B	2

A and B identify two different records with the same key 3.

Stable result

1	2	3A	3B

3A remains before 3B.

Unstable result

1	2	3B	3A

The two equal keys changed relative order.

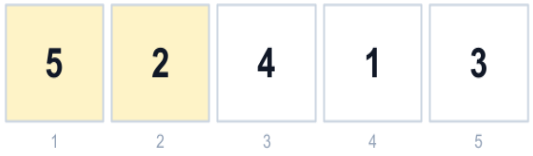
Bubble Sort and Insertion Sort are stable when implemented with strict comparison (>). Standard Selection Sort is not stable because it may swap a later equal item before an earlier one.

Bubble Sort: algorithm and optimized pseudocode

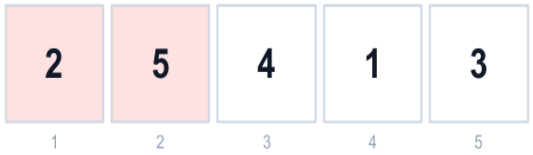
Repeatedly compare adjacent pairs and move the largest unsorted element to the right.

Idea

During each pass, adjacent elements A[j] and A[j+1] are compared. If A[j] > A[j+1], they are swapped. After pass k, the k largest elements are fixed at the right end.



Compare adjacent elements



Swap because 5 > 2

Optimized Bubble Sort pseudocode

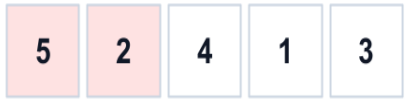
```
BubbleSort(A, n):
  for pass = 1 to n-1:
    swapped = false
    for j = 1 to n-pass:
      if A[j] > A[j+1]:
        swap A[j], A[j+1]
        swapped = true
    if swapped == false:
      break
```

The swapped flag is the code modification needed for O(N) best case on already sorted input.

Bubble Sort simulation: first pass

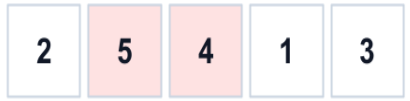
Example: [5, 2, 4, 1, 3]. The largest value reaches its final position.

Start



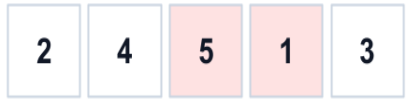
5 > 2 → swap

Step 1



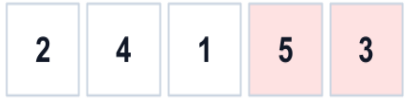
5 > 4 → swap

Step 2



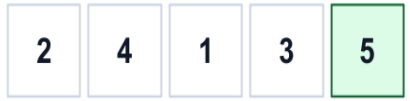
5 > 1 → swap

Step 3



5 > 3 → swap

End pass 1



5 fixed at right

Bubble Sort: case analysis

Optimized version with early stopping.

Case	Input pattern	Passes	Comparisons	Swaps	Time
Best	Already sorted	1	$N - 1$	0	$O(N)$
Average	Random order	$\approx N - 1$	$\Theta(N^2)$	Depends on inversions	$O(N^2)$
Worst	Reverse sorted	$N - 1$	$N(N-1)/2$	$N(N-1)/2$	$O(N^2)$

Space complexity: O(1)

Uses only temporary variables for swap and flag.

Stable: Yes

Equal elements are not swapped because condition is $A[j] > A[j+1]$, not $\geq$.

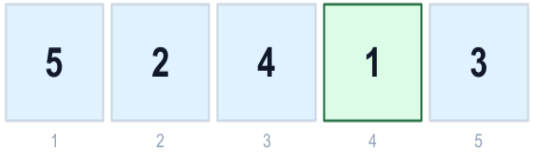
Without the swapped flag, best-case Bubble Sort remains $O(N^2)$, even on sorted input.

Selection Sort: algorithm and pseudocode

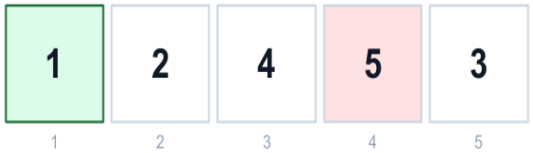
Repeatedly select the minimum value from the unsorted region and place it at the front.

Idea

For each index i, scan positions i to N to find the minimum. Then swap that minimum with A[i]. The sorted prefix grows by one element per pass.



Minimum is 1



Swap minimum into first position

Selection Sort pseudocode

```
SelectionSort(A, n):
  for i = 1 to n-1:
    minIndex = i
    for j = i+1 to n:
      if A[j] < A[minIndex]:
        minIndex = j
    if minIndex != i:
      swap A[i], A[minIndex]
```

Selection Sort always scans the unsorted region, so its comparison count does not improve on sorted input.

Selection Sort simulation

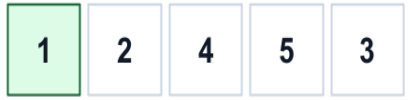
Each pass performs a full scan of the remaining unsorted region.

Pass 1



min=1; swap with 5

After pass 1



1 fixed

Pass 2



min=2; no swap

Pass 3



min=3; swap with 4

After pass 3



3 fixed

Pass 4 done



sorted

Selection Sort: case analysis

Comparison count is fixed; swap count can be low.

Case	Input pattern	Passes	Comparisons	Swaps	Time
Best	Already sorted	N − 1	N(N−1)/2	0 with conditional swap	O(N²)
Average	Random order	N − 1	N(N−1)/2	≤ N − 1	O(N²)
Worst	Reverse or adversarial	N − 1	N(N−1)/2	≤ N − 1	O(N²)

Space complexity: O(1)

Only a few variables are used: i, j, minIndex, temp.

Stable: No, standard version

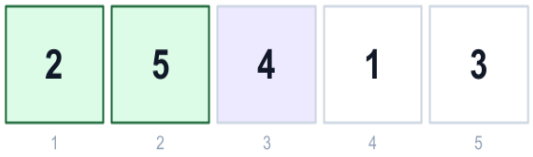
The final swap can move an equal key before another equal key. Stable selection sort is possible using shifting instead of swapping, but movement increases.

Insertion Sort: algorithm and pseudocode

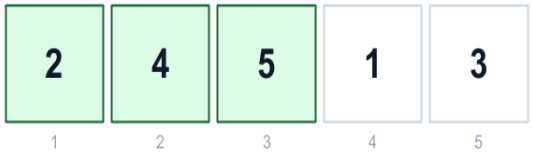
Insert each new key into the already sorted left side.

Idea

The subarray A[1..i−1] is maintained as sorted. The key A[i] is saved, larger values shift right, and the key is inserted into the open position.



key = 4; shift 5 right



Insert key before 5

Insertion Sort pseudocode

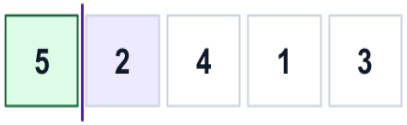
```
InsertionSort(A, n):
 for i = 2 to n:
  key = A[i]
  j = i - 1
  while j >= 1 and A[j] > key:
   A[j+1] = A[j]
   j = j - 1
  A[j+1] = key
```

The strict comparison A[j] > key keeps equal elements in their original relative order, so Insertion Sort is stable.

Insertion Sort simulation

The sorted prefix expands from left to right.

Start



key=2; shift 5

Insert 2



2 placed before 5

Insert 4



shift 5

Insert 1



shift 5,4,2

Insert 3



shift 5,4; sorted

Insertion Sort removes inversions by shifting. A nearly sorted array has few inversions, so it runs quickly.

Insertion Sort: case analysis

Adaptive behavior makes it efficient for already sorted or nearly sorted input.

Case	Input pattern	Passes	Comparisons	Shifts	Time
Best	Already sorted	N − 1	N − 1	0	O(N)
Average	Random order	N − 1	$\Theta(N^2)$	$\Theta(N^2)$	$O(N^2)$
Worst	Reverse sorted	N − 1	N(N−1)/2	N(N−1)/2	$O(N^2)$

Space complexity: O(1)

The key variable is extra; shifting happens inside the array.

Stable: Yes

Equal keys are not shifted when the comparison is strictly greater than the key.

Comparative complexity table

Summary of time, space, stability, and adaptiveness.

Algorithm	Best time	Average time	Worst time	Extra space	Stable?	Adaptive?
Bubble Sort optimized	O(N)	$O(N^2)$	$O(N^2)$	O(1)	Yes	Yes
Selection Sort standard	$O(N^2)$	$O(N^2)$	$O(N^2)$	O(1)	No	No
Insertion Sort	O(N)	$O(N^2)$	$O(N^2)$	O(1)	Yes	Yes

Academic wording: “Bubble and Insertion are stable and adaptive; standard Selection is in-place but not stable or adaptive.”

Adaptiveness means the algorithm can exploit existing order in the input. Stability matters when records have multiple fields, such as sorting students by marks while preserving previous roll-number order among equal marks.

Cheat sheet

Concise statements for Bubble, Selection, and Insertion Sort.

Algorithm	Core operation	Best case	Worst case	Stable?	Key note
Bubble	Adjacent compare + swap	O(N) with swapped flag	$O(N^2)$	Yes	Worst swaps = $N(N-1)/2$
Selection	Find min + place at left	$O(N^2)$	$O(N^2)$	No	Comparisons always $N(N-1)/2$
Insertion	Insert key into sorted prefix	O(N)	$O(N^2)$	Yes	Worst shifts = $N(N-1)/2$

Final takeaway

All three algorithms are comparison-based and in-place. Bubble and Insertion can be linear in the best case, but Bubble only achieves that with the early-stopping flag. Standard Selection Sort remains quadratic in all cases and is not stable.